

GRA-Living-Vaccine-Architecture: A Self-Learning Framework for Programmable Therapeutic Agents Based on Hierarchical Nullification and Swarm Intelligence

bitsoev oleg

<https://github.com/qqewq/GRA-Living-Vaccine-Architecture>

June 15, 2026

Abstract

We introduce a novel architectural framework for designing and simulating “living vaccines” — programmable therapeutic agents that autonomously adapt to arbitrary disease conditions. The framework is grounded in the Generalized Recursive Architecture (GRA) principles: hierarchical stability, nullification (controlled self-termination), and love-oriented swarm coordination. We provide a Domain-Specific Language (DSL) for specifying agent logic, a discrete-time simulator for signal environments (infection, inflammation, toxicity), and an evolutionary AI designer that optimizes agent parameters. Three nullification protocols (hard, soft, love-oriented) guarantee safety by forcing agents to shut down or transfer resources when stability constraints are violated. Experimental results on toy immunotherapy and cancer nullification scenarios demonstrate that the framework successfully balances therapeutic efficacy and safety. The entire implementation is open-source and intended as a conceptual platform for future research in synthetic biology and AI-driven medicine.

1 Introduction

Modern synthetic biology and cell therapy face a critical challenge: how to design therapeutic agents (e.g., CAR-T cells, engineered bacteria) that

are both effective against a wide range of diseases and intrinsically safe. Traditional approaches rely on external kill switches or fixed logic, which are brittle and cannot adapt to unforeseen conditions.

In parallel, the GRA (Generalized Recursive Architecture) project [1, 2, 3] has developed principles for building robust AI systems through hierarchical stability and nullification. This paper applies these principles to the domain of living therapeutics, creating a unified framework called **GRA-Living-Vaccine-Architecture**. Our contributions are:

- A formal DSL for specifying agent behavior, including signal processing, state updates, and nullification triggers.
- A multi-agent simulator with a dynamic signal environment.
- Three nullification protocols (hard, soft, love-oriented) that enforce safety at both individual and swarm levels.
- An evolutionary AI designer that automatically searches for optimal agent rules under given objectives.
- Open-source implementation and a LaTeX paper for reproducibility.

2 GRA Principles for Living Vaccines

We reinterpret three core GRA concepts for biological agents:

2.1 Hierarchical Stability

Each agent maintains an internal state (activation, energy) and the swarm as a whole computes a *Hierarchical Stability Rank* $R = \alpha(1 - a_v) + \beta(1 - e_v) + \gamma\ell$, where a_v is activation violation, e_v energy violation, and ℓ a love factor. If R drops below a threshold, nullification triggers.

2.2 Nullification

Three modes:

1. **Hard nullification:** immediate agent death (removal from simulation).
2. **Soft nullification:** agent deactivates but can later recover if signals improve.

3. **Love-oriented nullification:** before shutdown, agent distributes its remaining energy to neighbors and emits a love signal, enhancing swarm resilience.

2.3 Swarm Coordination

Agents can exchange energy and love signals. Love-oriented nullification implements a form of altruistic resource transfer, preventing the tragedy of the commons.

3 Architecture Overview

The framework consists of five layers (Fig. 1):

1. **Signals:** environment variables (infection_level, inflammation, etc.).
2. **Logic (DSL):** rules of the form IF condition THEN action.
3. **Agent State:** internal variables (activation, energy).
4. **Swarm:** multi-agent interactions and collective nullification.
5. **AI Design Core:** evolutionary optimizer that tunes rules and parameters.

[Diagram: Layers 0–5 with arrows]

Figure 1: Layered architecture of GRA-Living-Vaccine.

4 DSL and Simulator

The DSL (schema.yaml) allows declaring signals, state variables, rules, and stability constraints. Example rule:

```
- name: "activate_on_high_infection"
  condition:
    any_of:
      - all_of:
          - type: signal
            name: "infection_level"
```

```

        op: ">"
        value: 0.7
    action:
        type: "therapeutic_effect"
        params:
            delta_activation: 0.2
            cost_energy: 0.1

```

The simulator (`sim/`) implements a discrete-time loop where each agent evaluates its rules, updates state, and checks nullification. The environment dynamics are user-defined; we provide a simple periodic infection model.

5 AI Designer via Evolution

We implement an evolutionary algorithm (population size 10–12, generations 5–8) that mutates rule thresholds and action parameters. Fitness is defined as $\sum_t (1 - \text{infection}(t)) \cdot (1 - \text{activation}(t))$ over a 20-step horizon, penalizing nullification. Code is in `ai/designer.py`.

6 Experiments

6.1 Toy Immunotherapy

Agent from `simple_immunotherapy.yaml` was simulated for 30 steps. It activated when infection > 0.7 and energy > 0.3 . Soft nullification triggered when activation exceeded 1.0. The agent kept infection under control without premature nullification.

6.2 Cancer Nullification with Love Transfer

In the cancer scenario (`cancer_nullification.yaml`), agents used love-oriented nullification when toxicity > 0.8 . This caused a cascade of resource donation, maintaining population health. Hierarchical stability rank remained above 0.6 throughout.

6.3 Evolution Results

Running the AI designer for 8 generations produced agents with lower `cost_energy` and more precise infection thresholds, achieving 23% higher fitness than random baseline.

7 Conclusion and Future Work

We have presented a concrete implementation of GRA principles for living vaccine simulation. The framework is modular, extensible, and ready for integration with more realistic biological models (e.g., ODE-based pharmacokinetics). Future work includes connecting to digital twin platforms, adding reinforcement learning, and incorporating real omics data streams.

References

- [1] GRA Community. *GRA-Core-Unified-Hierarchical-Stability-Library*. GitHub, 2026.
- [2] GRA Community. *Love-Swarm-Nullification-Enhanced*. GitHub, 2026.
- [3] GRA Community. *GRA-Swarm-Subject*. GitHub, 2026.